实验一 Util

目标

熟悉如何在xv6环境下写代码

实验要求

完成sleep、pingpong、primes、find、xargs五个作业

作业一

- 为 xv6实现 UNIX 程序`sleep`，`sleep` 应该暂停用户指定的滴答数。

```
$make qemu
...
init: 启动sh
$sleep 10  (sleep命令的输入格式)
(一会儿什么也没发生)
$
```

- 时钟周期是 xv6 内核定义的时间概念，即定时器芯片两次中断之间的时间。
- 注意解决方案应该位于文件 `user/sleep.c`中。

作业二

- 使用 UNIX 系统调用通过一对`pipe`（每个方向一个）在两个进程之间“pingpong”一个字节。
- 父进程应向子进程发送一个字节；子进程应打印“：已收到 ping”，其中 是其进程 ID，将管道上的字节写入到父进程，然后退出；
- 父进程应该从子进程读取字节，打印“：收到 pong”，然后退出。
- 解决方案应该位于文件`user/pingpong.c`中。

作业三

- 使用`pipe`编写素数筛的并发版本。
- 这个想法源自 Unix `pipe`的发明者 Doug McIlroy。本页解释了如何操作。
- 解决方案应该位于文件`user/primes.c`中。

作业四

- 编写一个简单版本的 UNIX 查找程序：查找目录树中具有特定名称的所有文件。
- 解决方案应该位于文件`user/find.c`中。

作业五

- 编写一个简单版本的 UNIX xargs 程序：从标准输入读取行并为每行运行一个命令，将该行作为参数提供给命令。
- 解决方案应该位于文件`user/xargs.c`中。

实现说明

作业一

some hints

- 在开始编码之前，请阅读xv6 书的第 1 章。
- 查看`user/`中的一些其他程序（例如`user/echo.c`、`user/grep.c`和`user/rm.c`），了解如何获取传递给程序的命令行参数。
- 如果用户忘记传递参数，sleep 应该打印一条错误消息。
- 命令行参数作为字符串传递；您可以使用`atoi`将其转换为整数（请参阅 user/ulib.c）。
- 使用系统调用`sleep`。
- 请参阅`kernel/sysproc.c`以了解实现`sleep`系统调用的 xv6 内核代码（查找`sys_sleep`），请参阅`user/user.h`以了解可从用户程序调用`sleep`的 C 定义，以及`user/usys.s`以了解汇编代码从用户代码跳转到内核进行`sleep`。
- 确保`main`调用`exit()`以退出程序。
- 将你的`睡眠`程序添加到Makefile中的`UPROGS`中；完成此操作后，`make qemu`将编译您的程序，您将能够从 xv6 shell 运行它。

coding

- 创建`user/sleep.c`文件
- coding解决方案，注意参数个数的条件判断

```
if(argc<2){
    //函数体
}
```

- 系统调用sleep相关代码

```
int sleep(int);

uint64
sys_sleep(void)
{
  int n;
  uint ticks0;

  if(argint(0, &n) < 0)
    return -1;
  acquire(&tickslock);
  ticks0 = ticks;
  while(ticks - ticks0 < n){
    if(myproc()->killed){
      release(&tickslock);
      return -1;
    }
    sleep(&ticks, &tickslock);
  }
  release(&tickslock);
  return 0;
}
```

```
;位于文件user/usys.s中
sleep:
 li a7, SYS_sleep
 ecall
 ret
```

- 命令行参数作为字符串传递；使用`atoi`将其转换为整数

```
//user/ulib.c
int
atoi(const char *s)
{
  int n;

  n = 0;
  while('0' <= *s && *s <= '9')
    n = n*10 + *s++ - '0';
  return n;
}
```

作业二

some hints

- 使用`pipe`创建管道。
- 使用`fork`创建一个孩子。
- 使用`read`从管道读取数据，使用 `write`向管道写入数据。
- 使用`getpid`查找调用进程的进程 ID。
- 将程序添加到Makefile 中的`UPROGS`中。
- xv6 上的用户程序具有一组有限的可用库函数。可以在`user/user.h`中看到该列表；源代码（系统调用除外）位于`user/ulib.c`、`user/printf.c`和`user/umalloc.c`中

coding

- 创建`user/pingpong.c`
- 学习pipe管道的实现，举个简单例子

```
int p[2];
char *argv[2];
argv[0] = "wc";
argv[1] = 0;
pipe(p);
if (fork() == 0)
{
close(0); // 释放文件描述符 0
dup(p[0]); // 复制一个 p[0](管道读端)，此时文件描述符 0（标准输入）也
引用管道读端，故改变了标准输入。
close(p[0]);
close(p[1]);
exec("/bin/wc", argv); // wc 从标准输入读取数据，并写入到参数中的每
// 一个文件
}
else
{
close(p[0]);
```

```
write(p[1], "hello world\n", 12);
close(p[1]);
}
```

- getpid函数的使用方法

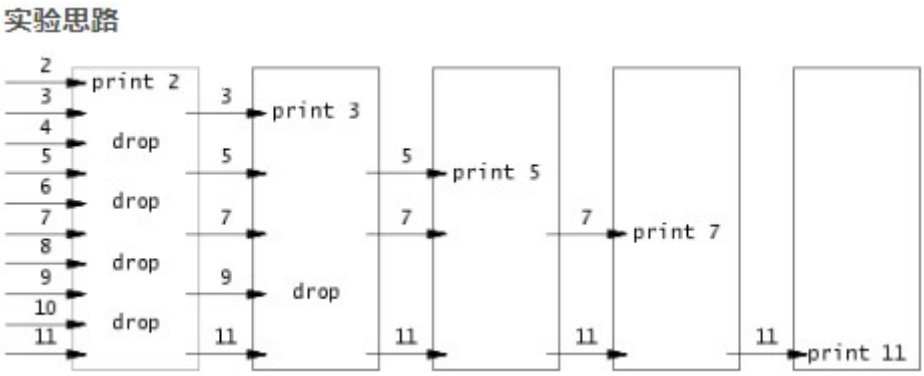
```
int getpid(void);
```

- 将程序添加到Makefile 中的`UPROGS`中，类似sleep

作业三

some hints

- 请小心关闭进程不需要的文件描述符，否则程序将在第一个进程达到 35 之前耗尽资源来运行 xv6。
- 一旦第一个进程达到 35，它应该等待整个管道终止，包括所有子进程、孙进程等。因此，主 primes 进程仅应在所有输出打印完毕以及所有其他 primes 进程退出后退出。
- 提示：当`pipe`的写入端关闭时， 读取返回零。
- 最简单的方法是直接将 32 位（4 字节）`int`写入管道，而不是使用格式化的 ASCII I/O。
- 仅在需要时在管道中创建进程。
- 将程序添加到Makefile 中的`UPROGS`中。
- 实验思路



- 由图可见，首先将数字全部输入到最左边的管道，然后第一个进程打印出输入管道的第一个数 2 ，并将管道中所有 2 的倍数的数剔除。接着把剔除后的所有数字输入到右边的管道，然后第二个进程打印出从第一个进程中传入管道的第一个数 3 ，并将管道中所有 3 的倍数的数剔除。接着重复以上过程，最终打印出来的数都为素数。

coding

- 创建文件user/primes.c
- 函数mapping：参考 xv6 手册第一章的 Pipes 部分，其中的例子演示了运行程序 wc ，由此受到启发，可以使用文件描述符重定向技巧，避免 xv6 的文件描述符限制所带来的影响。所以，学习示例代码，先写一个重定向函数 mapping ，虽然叫重定向，但感觉还是称为映射比较好，原理就是关闭当前进程的某个文件描述符，把管道的读端或者写端的描述符通过 dup 函数复制给刚刚关闭的文件描述符，再把管道描述符关闭，节约了资源。
- 函数primes：定义两个整型变量，存放从管道获取的数，定义管道描述符数组。从管道中读取数据，第一个数一定是素数，直接打印出来，然后创建另一个管道和子进程，子进程通过管道描述符映射，关闭不必要的文件描述符节约资源。再循环读取原管道中的数据，如果该数与原管道的第一个取模后不为 0 ，则再次写入刚刚创建的管道。父进程等待此子进程执行完毕，再次调用重定向函数关闭不必要的文件描述符，再递归调用 primes 函数重复以上筛选过程，即可将管道中所有素数打印出来。

- 函数main：首先创建文件描述符和创建一个管道，再创建一个子进程，子进程把管道的写端口描述符映射到原来的描述符标准输出 1 上。循环通过 write 系统调用把 2 到 35 写入管道。父进程等待子进程把数据写入完毕后，父进程把管道的读端口描述符映射到原来的描述符标准输入 0 上。在编写一个 primes 函数，调用此函数实现递归筛选的过程。

作业四

some hints

- 查看 user/ls.c 以了解如何读取目录。
- 使用递归允许 find 深入到子目录。
- 不要递归成“.” 和 ”..”。
- 文件系统的更改在 qemu 运行期间持续存在；运行一个干净的文件系统make clean，然后make qemu.
- 请注意，== 并不像 Python 中那样比较字符串。请改用 strcmp()。
- 将程序添加到Makefile 中的`UPROGS`中。

coding

- 根据提示，我们可以先阅读 `user/ls.c` 源码查看如何读目录。

```
//user/ls.c
char*
fmtname(char *path){}
//格式化文件或目录名称以进行显示的函数
//也就是把名字变成前面没有左斜杠 / ，仅仅保存文件名。

void
ls(char *path){}
//首先函数里面声明了需要用到的变量，包括文件名缓冲区、文件描述符、文件相关的结构体等等。其次使用open() 函数进入路径，判断此路径是文件还是文件名。
//如果是文件类型，则打印出文件信息，包括文件名，类型，Inode号，文件大小（单位为bytes）。如果是目录类型，则获取目录下的文件名，并依次输出文件信息。如果该文件还为路径，则循环遍历输出该路径下的文件信息
```

- 我们可以仿照上述内容，定义find()函数
-
  - 首先声明了文件名缓冲区、文件描述符和文件相关的结构体。
  - 其次试探是否能进入给定路径，然后调用系统调用获得一个已存在文件的模式，并判断其类型。如果该路径不是目录类型就报错。
  - 接着把绝对路径进行拷贝，循环获取路径下的文件名，与要查找的文件名进行比较，如果类型为文件且名称与要查找的文件名相同则输出路径
  - 如果是目录类型则递归调用 find() 函数继续查找。

作业五

some hints

- 使用`fork`和`exec`在每行输入上调用命令。在父级中使用`wait`来等待子级完成命令。
- 要读取单行输入，请一次读取一个字符，直到出现换行符 ('\n')。
- kernel/param.h 声明了 MAXARG，用于声明 argv 数组。
- 将程序添加到Makefile 中的`UPROGS`中。
- 文件系统的更改在 qemu 运行期间持续存。先`make clean`再`make qemu`

coding

- 创建文件 `user/xargs.c`.

- 理解UNIX中xargs命令。`xargs` 是一个用于构建和执行命令行的实用程序，通常用于处理通过管道传递的输入。`xargs` 从标准输入或者其他命令的输出中读取数据，并将其作为参数传递给指定的命令。
- 基本语法如下：

```
command | xargs [options] [command]
```

- `command`：要执行的命令。
- `[options]`：`xargs` 的选项，用于控制其行为。
- `[command]`：可选，如果省略，则默认使用 `echo` 命令。
- 注意题干说的“从标准输入读取行并为每行运行一个命令，”，这里的标准输入即0，和管道pipe的0是同一个引用。即将基本语法中的管道的读入直接使用标准输入的0替代即可

```
read(0, buf+m, 1)
```